Avance de Proyecto Final 2

Sistema de Matrícula

Curso: Herramientas de Desarrollo (100000S66T)
Periodo: 2026-II
Semana: 11
Version: v2.0.0
Proyecto: Sistema de Matrícula
Repositorio: github.com/Rodrigo2326/sistema-matricula

Integrantes:
Ccahuana Huillca Giancarlo
Aparcana Mamani Rodrigo Alonso
Arnold Jhuncor Díaz Silva

1. Objetivo del avance

El APF1 dejó el proyecto versionado y con el flujo fundamental de Git funcionando. Este segundo avance tiene un objetivo distinto: convertir ese repositorio en un **entorno de trabajo colaborativo organizado y trazable**.

En términos concretos, eso significó dejar de integrar cambios directamente sobre `main` y pasar a un flujo en el que toda modificación entra por Pull Request, es revisada por otro integrante antes de integrarse, y queda relacionada con una tarea del tablero. El producto siguió creciendo —se agregaron tres funcionalidades— pero lo que este avance pretende demostrar es el **proceso** con el que se agregaron.

2. Continuidad del proyecto

Se mantiene el mismo proyecto del APF1: una aplicación web para el registro y consulta de matrículas de estudiantes por ciclo académico, que se ejecuta en el navegador sin servidor ni instalación y guarda la información en el almacenamiento local.

La continuidad es verificable en el historial: el avance parte del commit `83d4533`, que cerró el APF1, y no hay en el repositorio ninguna carpeta paralela ni copia del proyecto. Todo el crecimiento está en la misma línea de historia.

3. Evolución funcional del producto

El APF1 entregó el registro de matrículas, el listado con contador, la búsqueda, los filtros por carrera y ciclo, el ordenamiento por columnas, la edición, la validación, la exportación a CSV y el resumen por carrera.

El APF2 agregó **tres funcionalidades nuevas, una por integrante**.

3.1. Catálogo de cursos con control de vacantes

Hasta el APF1 el curso a matricular se escribía a mano en un campo de texto libre. Eso tenía dos consecuencias: el mismo curso podía quedar registrado de varias formas distintas, y —más grave— no existía ninguna noción de capacidad. Se podía matricular a cuarenta estudiantes en un aula de treinta.

Se incorporó una sección de **Catálogo de cursos** donde se da de alta cada curso con su código, nombre, carrera, ciclo y número de vacantes. El listado muestra la ocupación en formato `matriculados / vacantes` y una etiqueta de estado que indica cuántas vacantes quedan libres o si el cupo se agotó.

En el formulario de matrícula, el campo de curso pasó de ser un texto libre a un **selector alimentado por el catálogo**, filtrado por la carrera y el ciclo elegidos, que muestra las vacantes restantes de cada curso y deshabilita los que ya están llenos. La validación rechaza la matrícula cuando el cupo se agotó, de modo que la restricción no depende solo de la interfaz.

Dos reglas adicionales protegen la coherencia de los datos: no se puede eliminar un curso que tenga estudiantes matriculados, ni reducir sus vacantes por debajo de la ocupación que ya registra.

3.2. Constancia de matrícula imprimible

Una matrícula registrada solo existía como una fila de una tabla. El estudiante no tenía forma de obtener un comprobante.

Se agregó un botón **Constancia** en cada fila del listado que abre una vista con los datos del estudiante, el curso, el código del curso, la carrera, el ciclo y la fecha de matrícula, además de la fecha de emisión. La vista se puede imprimir o guardar como PDF desde el propio navegador.

La impresión se resolvió con una regla `@media print` en la hoja de estilos, de modo que al imprimir se ocultan el encabezado, los formularios, la barra de herramientas y los propios botones de la constancia, y queda únicamente la hoja. Es una decisión relevante: hacerlo por CSS en lugar de ocultar elementos

con JavaScript significa que la aplicación no cambia de estado para imprimir.

3.3. Panel de ocupación por curso

El catálogo muestra la ocupación curso por curso, pero leer veinte filas de números no da una lectura rápida de la situación general.

Se agregó un botón **Ver ocupación** que despliega un panel con una barra de progreso por curso, con un semáforo según el llenado: verde por debajo del 70 %, ámbar entre 70 % y 99 %, y rojo al 100 %. Encima del panel, un aviso indica cuántos cursos quedaron sin vacantes.

El panel reutiliza las funciones `contarOcupacion()` y `vacantesLibres()` del módulo del catálogo en lugar de recalcular la ocupación por su cuenta. Esto importa más de lo que parece y se explica en la sección 9.

3.4. Organización del código

Cada funcionalidad se desarrolló en su propio archivo, de modo que las tres ramas no compitieran por las mismas líneas:

Archivo	Responsabilidad	Líneas
`js/app.js`	Registro, listado, búsqueda, filtros y validación	430
`js/cursos.js`	Catálogo de cursos y control de vacantes	332
`js/constancia.js`	Constancia de matrícula imprimible	60
`js/ocupacion.js`	Panel de ocupación por curso	71

4. Organización del equipo

Integrante	Usuario de GitHub	Rol	Aporte en el APF2
Aparcana Mamani Rodrigo Alonso	@Rodrigo2326	Responsable de repositorio y documentación	Catálogo de cursos, integración de las ramas, release e informe
Ccahuana Huillca Giancarlo	@soulahrikermette-hub	Responsable funcional	Constancia de matrícula y revisión del PR #2
Arnold Jhuncor Díaz Silva	@gamergggpro123-lgtm	Responsable de calidad y automatización	Panel de ocupación por curso

El reparto se hizo asignando a cada integrante **una funcionalidad completa**, no fragmentos de una misma. La razón es que así cada uno tiene que entender su parte de punta a punta —interfaz, lógica y estilos— y puede responder por ella en la sustentación.

5. Herramienta de colaboración del equipo

El equipo utilizó **Microsoft Teams** como herramienta de colaboración.

Se estableció una **reunión de coordinación diaria de lunes a viernes, de 10:00 a 10:30 horas**, es decir cinco reuniones de media hora por semana. En cada reunión cada integrante expuso el avance de su rama, se revisaron los Pull Requests pendientes de aprobación y se repartieron las tarjetas del tablero.

El canal del equipo en Teams se utilizó además para resolver dudas puntuales entre reuniones y, sobre todo, para acordar el **orden de integración de las ramas**. Esto último resultó necesario porque las tres funcionalidades dependían del catálogo de cursos, y sin un acuerdo explícito sobre qué se integraba primero los conflictos de fusión habrían aparecido por sorpresa.

La reunión diaria corta, en lugar de una reunión larga semanal, fue una decisión deliberada: con tres personas trabajando en ramas que se tocan entre sí, media hora diaria detecta los choques antes de que se vuelvan costosos.

6. Gestión de actividades y del proyecto

Las actividades del avance se gestionaron con **GitHub Issues**, cada una con responsable asignado, etiqueta de tipo y el hito **APF2 - Semana 11**, con fecha límite al 14 de noviembre de 2026.

Se definieron diez tarjetas:

#	Actividad	Responsable	Estado
3	Catálogo de cursos con control de vacantes	Rodrigo Aparcana	Finalizado
4	Constancia de matrícula imprimible	Giancarlo Ccahuana	Finalizado
5	Panel de ocupación por curso	Arnold Díaz	Finalizado
6	Revisar los Pull Requests antes de integrarlos	Los tres	En proceso
7	Publicar la release v2.0.0	Rodrigo Aparcana	Finalizado
8	Informe APF2, README y evidencias	Rodrigo Aparcana	Finalizado
9	Coordinar el equipo en Microsoft Teams	Los tres	En proceso
10	Integrar las ramas y resolver el conflicto	Rodrigo Aparcana	Finalizado
11	Probar las funcionalidades y registrar resultados	Arnold Díaz	Pendiente
12	Redactar la evolución funcional del informe	Giancarlo Ccahuana	Pendiente

Las etiquetas utilizadas fueron `funcionalidad`, `revision`, `documentacion` y `gestion`, todas bajo la etiqueta común `apf2`.

6.1. Relación entre una tarea del tablero y el cambio en el repositorio

Esta relación es trazable en los dos sentidos. Tomando como ejemplo la tarjeta **#3, Catálogo de cursos con control de vacantes**:

1. La tarjeta describe el objetivo y seis criterios de aceptación.
2. El trabajo se hizo en la rama `feature/catalogo-cursos`.
3. Los commits `35cdac0`, `e1074f8` y `5c61153` implementan esos criterios.
4. El Pull Request #2 agrupa esos commits y referencia la tarjeta.
5. Al integrarse el PR, la tarjeta se cerró con un comentario que enlaza los commits concretos.

La misma cadena existe para las otras dos funcionalidades:

Tarjeta	Rama	Commits	Pull Request
#3	`feature/catalogo-cursos`	`35cdac0`, `e1074f8`, `5c61153`	#2
#4	`feature/constancia-matricula`	`3c11fcd`, `55ca323`	#13
#5	`feature/ocupacion`	`deba101`, `d73c1c6`	#14

7. Flujo de trabajo con ramas y Pull Requests

La convención de ramas se mantuvo del APF1: `main` como versión estable, `feature/<nombre>` para funcionalidades y `fix/<nombre>` para correcciones.

La diferencia con el avance anterior es que **ninguna rama se integró directamente**. Las tres pasaron por Pull Request:

PR	Título	Autor	Creado	Integrado	Líneas
#2	Agregar catálogo de cursos con control de vacantes	Rodrigo Aparcana	21/10	24/10	+488 / –11
#13	Agregar constancia de matrícula imprimible	Giancarlo Ccahuana	23/10	24/10	+647 / –5
#14	Agregar panel de ocupación por curso	Arnold Díaz	24/10	24/10	+264 / –2

Las fusiones se hicieron conservando el punto de integración, de modo que el gráfico del historial muestra explícitamente dónde entró cada funcionalidad.

> **Nota sobre el PR #14.** Los commits del panel de ocupación son de Arnold > Díaz, pero el Pull Request fue abierto por Rodrigo Aparcana para poder cerrar > la integración dentro del plazo del avance. Queda registrado aquí por > transparencia.

8. Revisión de código antes de integrar

Este es el punto donde el avance produjo su evidencia más útil.

El **Pull Request #2** fue revisado por Giancarlo Ccahuana, que lo aprobó pero dejó **dos comentarios sobre líneas concretas del código**. Los dos resultaron ser defectos reales:

Primera observación, sobre `js/cursos.js`. Al renombrar un curso desde el catálogo, las matrículas ya registradas seguían mostrando el nombre anterior. La causa era que cada matrícula guarda una copia del nombre del curso —para que el listado, el CSV y la constancia no dependan de que el curso siga existiendo en el catálogo— y el código solo propagaba el cambio de código, no el de nombre.

Segunda observación, sobre `js/app.js`. La validación de matrículas duplicadas comparaba el curso por su nombre y no por su código. Como dos carreras —o dos ciclos de la misma carrera— pueden ofrecer cursos que se llaman igual, un estudiante que se matriculara en «Práctica Preprofesional» del ciclo IX quedaba bloqueado al intentar matricularse en la del ciclo X, siendo dos cursos distintos.

Ambos defectos se corrigieron en el commit `5c61153` antes de integrar el PR, y se respondió en el hilo de cada comentario explicando la corrección y cómo se verificó.

El **Pull Request #13** fue revisado por Rodrigo Aparcana, que verificó en navegador el llenado de los nueve campos de la constancia, el comportamiento al imprimir y el cierre de la vista, y dejó una observación menor sobre el formato del archivo.

La conclusión que el equipo extrae de esto es que la revisión de código no fue un trámite para cumplir con el requisito: encontró errores que el autor del código no había visto, y que no se habrían detectado probando la aplicación por encima.

9. Conflicto de fusión y su resolución

Al integrar la tercera rama se produjo un conflicto real, no provocado artificialmente.

Cómo se originó. La rama `feature/ocupacion` se creó a partir de `feature/catalogo-cursos`, por lo que no contenía el trabajo de la constancia. Mientras tanto, `main` ya había integrado el PR #13. Ambas funcionalidades agregaron reglas nuevas al final de `css/estilos.css`, en el mismo punto del archivo: justo después de la última regla del catálogo de cursos. Git no tiene forma de decidir cuál de los dos bloques va primero y detuvo la fusión.

Criterio de resolución. Los dos bloques son necesarios y no se superponen: uno da estilo a la constancia imprimible y el otro a las barras de ocupación. No había nada que elegir ni descartar, solo que ordenar. Se conservaron ambos, dejando primero el de la constancia —que ya estaba en `main`— y después el de ocupación, de modo que el archivo mantuviera el mismo orden en que las

funcionalidades se integraron.

Un error cometido durante la resolución. En un primer intento se perdió la llave de cierre del bloque `@media print` de la constancia, lo que dejó todas las reglas del panel de ocupación anidadas dentro de la regla de impresión. El efecto era sutil: la aplicación no mostraba ningún error, pero las barras de ocupación perdían el color en pantalla y solo se habrían visto al imprimir. El problema se detectó al volver a probar la aplicación en el navegador, no al leer el código.

La resolución definitiva se rehízo de forma verificable: se tomó el archivo de `main` como base, se anexó únicamente el bloque del panel de ocupación y se comprobó que el número de llaves de apertura y de cierre coincidiera (96 y 96) y que el bloque anexado no quedara anidado dentro de ninguna regla previa.

Una corrección adicional. Al integrar se detectó que la rama de ocupación cargaba `js/ocupacion.js` antes que `js/cursos.js`, del que depende. Funcionaba por casualidad, porque en la carga inicial ese archivo no referencia nada del catálogo —solo al pulsar el botón, cuando todo ya está definido—, pero el orden correcto es por dependencia:

```
app.js -> cursos.js -> constancia.js -> ocupacion.js
```

La evidencia completa del conflicto está en `docs/evidencias-apf2/`, en los archivos `conflicto-01-deteccion.txt`, `conflicto-02-marcadores.txt` y `conflicto-03-resolucion.txt`.

10. Release del avance

Con las tres funcionalidades integradas en `main` se publicó la versión **v2.0.0**, etiquetada sobre el commit de integración del PR #14.

Se eligió el salto de versión mayor porque el avance introduce un cambio que rompe la compatibilidad con la versión anterior: el curso de una matrícula pasó de ser texto libre a una referencia al catálogo. Las matrículas creadas con la versión `v1` siguen funcionando —el código las reconoce comparando nombre, carrera y ciclo— pero el formato de los datos cambió.

11. Verificación funcional

Las tres funcionalidades se probaron en conjunto sobre la rama integrada, ejecutando la aplicación con `python -m http.server` y operándola en el navegador.

Comprobación	Resultado
Catálogo intacto tras registrar matrículas	4 cursos
Ocupación reflejada en el catálogo	`1 / 30`
Curso con cupo agotado	`Sin vacantes`
Matrícula rechazada en curso lleno	mensaje de validación
Edición de matrícula en curso lleno	permitida
Panel de ocupación, curso al 100 %	`2 / 2 (100%)` en rojo
Umbral del 70 %	`7 / 10 (70%)` en ámbar
Aviso de cursos sin vacantes	`1 curso(s) sin vacantes.`
Constancia: estudiante, curso y código	correctos
Constancia al imprimir	oculta la aplicación, deja la hoja
Búsqueda y filtros del APF1	operativos
Resumen por carrera del APF1	operativo

Comprobación	Resultado
Errores de consola	0

12. Participación del equipo

Commits del avance, desde el cierre del APF1:

Integrante	Commits del APF2	Acumulado del proyecto
Aparcana Mamani Rodrigo Alonso	7	48
Ccahuana Huillca Giancarlo	2	8
Arnold Jhuncor Díaz Silva	2	7

El reparto de commits sigue siendo desigual, y conviene decirlo con precisión en lugar de disimularlo. De los siete commits de este avance atribuidos al responsable de repositorio, tres corresponden a las fusiones de los Pull Requests y uno a la documentación; el trabajo de desarrollo propiamente dicho está más repartido de lo que sugiere el conteo bruto. Aun así, el equilibrio sigue siendo una deuda del equipo, y es el punto a corregir en el APF3.

Lo que sí mejoró respecto del APF1 es que **cada integrante tiene una funcionalidad propia, íntegra y rastreable**, en lugar de contribuciones sueltas.

13. Reflexión sobre la dinámica de trabajo

Tres cosas funcionaron y dos no.

Funcionó repartir archivos, no funciones. Asignar a cada integrante su propio archivo `.js` evitó que las tres ramas compitieran por las mismas líneas de `app.js`. El único conflicto del avance apareció en la hoja de estilos, que es compartida por naturaleza, y fue trivial de resolver porque los bloques no se superponían.

Funcionó la revisión de código con comentarios sobre líneas concretas. Los dos defectos que encontró la revisión del PR #2 no se habrían detectado probando la aplicación: los dos requieren un escenario específico —renombrar un curso ya matriculado, o dos cursos homónimos en ciclos distintos— que nadie prueba por casualidad.

Funcionó la reunión diaria corta. Con ramas que dependen unas de otras, media hora diaria sirve para acordar el orden de integración. Una reunión semanal larga habría llegado tarde.

No funcionó demorar la integración del PR #2. El catálogo de cursos quedó aprobado varios días antes de integrarse a `main`. Mientras tanto, los otros dos integrantes no tenían una base actualizada de dónde partir: uno optó por ramificar desde la rama del catálogo y el otro arrancó desde `main`, sin el catálogo, lo que lo llevó a reescribir dos funciones que ya existían. Esas funciones duplicadas habrían roto el conteo de ocupación al integrarse. La lección es concreta: **un Pull Request aprobado debe integrarse de inmediato**, porque mientras siga abierto bloquea a quien depende de él.

No funcionó confiar en la lectura del código para validar la resolución del conflicto. El error del bloque `@media print` pasó la inspección visual sin problemas y solo apareció al abrir la aplicación. Toda resolución de conflicto debe cerrarse ejecutando la aplicación, no leyendo el diff.

14. Conclusiones

El avance cumplió su objetivo: el repositorio pasó de ser un lugar donde se guardan commits a ser un entorno de trabajo con un flujo definido. Toda modificación del producto en este periodo entró por una rama, fue propuesta mediante un Pull Request, fue revisada por otro integrante y quedó asociada a una tarea con responsable y estado.

La evidencia más valiosa del avance no son las tres funcionalidades nuevas, sino los dos defectos que la revisión de código detectó antes de que llegaran a `main`. Eso es lo que distingue un flujo colaborativo real de uno simulado para la entrega.

15. Próximos pasos (APF3)

- Registrar incidencias como issues de tipo error y cerrar al menos una con la evidencia del commit que la resolvió.
- Configurar un flujo de integración continua que valide automáticamente la sintaxis de los archivos JavaScript y la estructura del HTML.
- Configurar un flujo de entrega continua que publique la aplicación al integrar en `main`.
- Desplegar una versión de prueba en una plataforma en la nube.
- Equilibrar el reparto de commits entre los tres integrantes.